Computer Networks Lab Report

Assignment 1: Error Detection (Checksum and CRC)

Name: Sampad De
Class: BCSE-III
Roll No.: 002410501025

August 19, 2026

Course

Computer Networks Lab, CO1: Design and implement error detection techniques within a simulated network environment.

Problem Statement

Design and implement an error-detection module supporting two schemes, the 16-bit Internet checksum and Cyclic Redundancy Check (CRC), over a simulated UDP link, and empirically evaluate their error-detection reliability and computational cost against four generator polynomials (CRC-8, CRC-10, CRC-16, CRC-32) and four categories of injected transmission error.

1 Design

1.1 Purpose

The system emulates a simplified Ethernet-style data-link layer. A sender reads a file, packages it into fixed-size frames, appends a trailer computed by one of five detection schemes, and transmits each frame over a local UDP socket. A configurable error-injection module corrupts a controllable subset of frames before they leave the sender. The receiver listens on the same socket, recomputes the check value over each received frame, compares it against the trailer, and logs whether the frame was accepted or rejected. A separate, socket-free benchmarking program repeats this comparison offline across hundreds of thousands of randomly generated frames so that detection rates and timing can be measured with statistical confidence, independent of network jitter.

1.2 Frame Structure (`info.h`)

```
#define PAYLOAD_SIZE 44

typedef struct frame_header {
    unsigned char SOURCE_MAC[6];
    unsigned char DEST_MAC[6];
    uint16_t length;
} frame_header; // 14 bytes
```

```
typedef struct frame_trailer {
    uint32_t crc_send;
} frame_trailer; // 4 bytes

typedef struct Frame {
    frame_header header;
    unsigned char payload[PAYLOAD_SIZE];
    frame_trailer trailer;
} frame;
```

With a 44-byte payload this places every frame's total on-wire size within the Ethernet IEEE 802.3 payload envelope (46–1500 bytes is the underlying frame's data field; here the simulated frame itself, header+payload+trailer, is used directly as the unit exchanged over UDP). I verified the in-memory layout with `sizeof()`: due to 4-byte alignment of the `uint32_t` trailer, the compiler inserts 2 bytes of padding after the 58-byte header+payload region, so `sizeof(frame)` is 64 bytes even though only 14+44+4 = 62 bytes are semantically meaningful. This is a benign artifact of struct alignment and does not affect correctness, since both sender and receiver use the same compiled struct layout.

1.3 Error-Detection Schemes (`error_injection.c`)

16-bit Internet Checksum. The header and payload (everything except the trailer) are split into 16-bit words, summed with one's-complement (end-around-carry) arithmetic, and the final sum is bitwise complemented:

```
uint16_t compute_checksum(uint8_t* data, size_t len) {
    uint32_t sum = 0;
    for (size_t i = 0; i < len; i += 2) {
        uint16_t word = (data[i] << 8) | ((i + 1 < len) ? data[i + 1] : 0);
        sum += word;
        if (sum > 0xFFFF) sum = (sum & 0xFFFF) + 1; // end-around carry
    }
    return (uint16_t)(~sum);
}
```

CRC (generic, bit-by-bit). A single width-parameterised routine implements all four CRC variants by shifting each byte into a register and XOR-ing with the generator polynomial whenever the top bit is set:

```
uint64_t calculate_crc(const uint8_t* data, size_t length,
                       uint64_t polynomial, unsigned width) {
    uint64_t crc = 0;
    uint64_t top_bit = 1ULL << (width - 1);
    uint64_t mask = (1ULL << width) - 1;
    for (size_t i = 0; i < length; i++) {
        crc ^= (uint64_t)data[i] << (width - 8);
        for (size_t j = 0; j < 8; j++) {
            crc = (crc & top_bit) ? (crc << 1) ^ polynomial : (crc << 1);
            crc &= mask;
        }
    }
```

```
    return crc;
}
```

Generator polynomials. The four CRC widths are configured as follows:

Algorithm	Width	Generator (Hex)	Typical use
CRC-8	8	`0xD5`	General / Bluetooth
CRC-10	10	`0x289`	General / Telecom
CRC-16	16	`0x8005`	USB
CRC-32	32	`0x04C11DB7`	Ethernet IEEE 802.3

Verification note: I re-derived each hex constant directly from the polynomial given in the assignment (e.g. $x^8+x^7+x^6+x^4+x^2+1$, dropping the implicit leading term). CRC-8 (`0xD5`), CRC-16 (`0x8005`) and CRC-32 (`0x04C11DB7`) all match exactly. For CRC-10, the assignment's polynomial $x^{10}+x^9+x^5+x^4+x+1$ (the standard ATM/ITU-T CRC-10, confirmed against the original IBM patent describing this exact generator) works out to `0x233`, whereas the code uses `0x289` – a different degree-10 polynomial. This does not break the experiment (sender and receiver both use the same constant, so detection is internally consistent, and the code still analyses a valid degree-10 CRC), but the CRC-10 results below should be read as characterising the polynomial actually implemented rather than the textbook ATM constant. This is worth fixing in the source before final submission if an exact match to the assignment's CRC-10 is required.

1.4 Error Injection Module (`info.c`)

- `inject_single_bit_error()` – flips exactly 1 random bit within the header+payload region.
- `inject_iso_double_bit_error()` – flips 2 distinct random bits (a shared "already flipped" tracking array guarantees they are never the same bit, so no error can silently cancel itself).
- `inject_odd_num_error()` – flips 1, 3, 5, or 7 random distinct bits, chosen uniformly.
- `inject_burst_error(len, prob)` – picks a random contiguous window of `len` bits and flips each bit in that window independently with probability `prob` (I used `prob = 0.7` throughout, following the sender's own default).
- `inject_random_bit_error(n)` – the shared primitive underlying all of the above; flips `n` guaranteed-distinct random bits.

Both the sender/receiver programs (interactive, over UDP) and the benchmark program (`test_eval.c`, offline, no sockets) link against the exact same `info.c` / `error_injection.c` translation units, so the statistics below characterise the identical code path that runs live between `sender` and `receiver`.

1.5 Frame I/O Format

Sender usage: `./sender <filename> <ip>`, followed by an interactive prompt for the detection scheme (0–4) and, for the demo, an error-injection type. The receiver usage is `./receiver <out_filename>`, prompted for the same detection scheme. Every accepted or rejected frame is appended to the output file as a delimited block, e.g. for the supplied `test.txt` test string with the checksum scheme and no error injection:

```
<START>
STATUS: VALID
```

```
Length: 41 bytes
Ab1!@#9()_+[]{}|;:',./<>?~=-␣`aBcDeFgHiJk
<END>
```

I reproduced this myself by compiling and running `sender`/`receiver` over loopback UDP with `test.txt` as input, and confirmed the receiver logs an identical `STATUS: VALID` block with the payload reconstructed byte-for-byte.

2 Test Cases

#	Error Type	Expected outcome
T1	None	All five schemes: 0% rejection (no false positives).
T2	Single bit	All five schemes: 100% detection (guaranteed by theory).
T3	Isolated double bit	Checksum shows a measurable miss rate from carry cancellation; all CRCs detect substantially more reliably.
T4	Isolated double bit	CRC-8 (256 states) misses more often than CRC-16/CRC-32 (65,536 / 4.3×10^9 states) due to a higher birthday-style collision probability.
T5	Odd number of errors	Polynomials divisible by $(x+1)$ (CRC-8, CRC-16, CRC-32 here) guarantee detection; polynomials without that factor can miss a small fraction.
T6	Burst, length $L \leq r$	Detection probability = 1.0 (guaranteed by the CRC degree-r bound).
T7	Burst, length $L = r+1$	Detection probability $\approx 1 - (1/2)^{r-1}$.
T8	Burst, length $L > r+1$	Detection probability $\approx 1 - (1/2)^{r}$.

3 Results

Dataset Summary

All figures below come from `test_eval.c`, compiled and executed directly from the submitted source (`gcc -O2`, no source modification). Each error model below was evaluated over 200,000 independently randomised 64-byte frames, for a total of 1,200,000 simulated frame transmissions across the six error models. I re-ran the whole benchmark myself rather than reusing pre-computed numbers, so the values below reflect a fresh, independently reproducible execution rather than a cached result; because the RNG is seeded from wall-clock time, exact figures will fluctuate by a few hundredths of a percent between runs, but the qualitative pattern (below) is stable and matches theory.

3.1 Error Detection and Overlap Analysis

For every pair (Checksum, CRC-X), each of 200,000 random frames was corrupted once and both schemes were evaluated on the identical corrupted frame, so the four outcome categories are mutually exclusive per trial.

Error Model	Scheme	Both Detect	Checksum Only	CRC Only	Miss Both
Single Bit	CRC-8	100.0000%	0.0000%	0.0000%	0.0000%
	CRC-10	100.0000%	0.0000%	0.0000%	0.0000%
	CRC-16	100.0000%	0.0000%	0.0000%	0.0000%
	CRC-32	100.0000%	0.0000%	0.0000%	0.0000%
Isolated Double Bit	CRC-8	96.1615%	0.8555%	2.9830%	0.0000%
	CRC-10	97.0170%	0.0000%	2.9830%	0.0000%
	CRC-16	97.0170%	0.0000%	2.9830%	0.0000%
	CRC-32	97.0170%	0.0000%	2.9830%	0.0000%
Odd Number of Errors	CRC-8	99.9035%	0.0000%	0.0965%	0.0000%
	CRC-10	99.8445%	0.0590%	0.0960%	0.0005%
	CRC-16	99.9035%	0.0000%	0.0965%	0.0000%
	CRC-32	99.9035%	0.0000%	0.0965%	0.0000%
Burst (Length 12)	CRC-8	99.6365%	0.3635%	0.0000%	0.0000%
	CRC-10	99.9185%	0.0815%	0.0000%	0.0000%
	CRC-16	99.9990%	0.0010%	0.0000%	0.0000%
	CRC-32	100.0000%	0.0000%	0.0000%	0.0000%
Burst (Length 32)	CRC-8	99.6165%	0.3825%	0.0010%	0.0000%
	CRC-10	99.8920%	0.1070%	0.0010%	0.0000%
	CRC-16	99.9975%	0.0015%	0.0010%	0.0000%
	CRC-32	99.9990%	0.0000%	0.0010%	0.0000%
Gamma Burst (shape=2, scale=20)*	CRC-8	99.4030%	0.3905%	0.0010%	0.2055%
	CRC-10	99.6985%	0.0950%	0.0010%	0.2055%
	CRC-16	99.7905%	0.0030%	0.0010%	0.2055%
	CRC-32	99.7935%	0.0000%	0.0010%	0.2055%

*Added metric, not required by the assignment: burst lengths drawn from a Gamma(shape=2, scale=20) distribution (via Marsaglia–Tsang) rather than a fixed length, to approximate the short, frequent, heavy-tailed bursts typical of real impulse noise instead of a single worst-case burst length.

Collapsing the table above into each scheme's own overall miss rate per error model (i.e. regardless of what the other scheme did) gives a clearer reliability ranking:

Error Type	Checksum	CRC-8	CRC-10	CRC-16	CRC-32
Single Bit	100.0000%	100.0000%	100.0000%	100.0000%	100.0000%
Isolated Double Bit	97.0170%	96.1615%	97.0170%	97.0170%	97.0170%
Odd Numbered	99.9035%	99.9035%	99.8450%	99.9035%	99.9035%
Burst (L=32)	99.9990%	99.6175%	99.8930%	99.9985%	99.9990%

(Detection rate = 100% − that scheme's own miss rate, i.e. "Checksum Only" + "Both Detect" for the checksum column, and "CRC Only" + "Both Detect" for each CRC column.)

3.2 Computational Efficiency

Measured with `clock_gettime(CLOCK_MONOTONIC)` over 5,000,000 repeated computations on a single fixed 60-byte frame:

Scheme	Total Time (s)	Avg. Time / Frame (ns)	Throughput (MB/s)
Checksum	0.1811	36.23	1579.57
CRC-8	3.3498	669.96	85.41
CRC-10	4.9828	996.57	57.42
CRC-16	5.0113	1002.26	57.09
CRC-32	3.3586	671.73	85.18

The checksum is roughly 18–28× faster than the CRC variants, since it processes data 16 bits at a time using native CPU addition, whereas the CRC implementation here is bit-serial (8 shift/XOR steps per byte). Note CRC-10 and CRC-16 are noticeably slower than CRC-8 and CRC-32 in this run despite similar per-byte work – both use narrower (10/16-bit, non-native-width) integer masking inside the inner loop, which appears to compile slightly less efficiently than the byte- and word-aligned CRC-8/CRC-32 cases on this machine; a lookup-table implementation (see Section 6) would remove this asymmetry entirely.

3.3 Boundary Cases: Concrete Examples

The following genuine (non-fabricated) collision examples were found by exhaustively scanning up to 1,000,000 randomly corrupted frames for each case, using CRC-8 as the representative narrow CRC:

Case 1 – Detected by both. Original checksum `0x34D9` → corrupted `0x94DC` (changed); original CRC-8 `0xC3` → corrupted `0xB0` (changed). Both schemes correctly reject the frame.

Case 2 – Detected by checksum, missed by CRC-8. Original checksum `0xA586` → corrupted `0xE616` (changed, detected); original CRC-8 `0xA0` → corrupted `0xA0` (unchanged – a genuine 8-bit collision). This is a direct demonstration of CRC-8's narrow 256-value remainder space allowing two different frames to alias to the same remainder.

Case 3 – Detected by CRC-8, missed by checksum. Original checksum `0x4368` → corrupted `0x4368` (unchanged – a checksum collision, e.g. a compensating pair of bit flips across a 16-bit word boundary); original CRC-8 `0x7A` → corrupted `0x20` (changed, detected).

4 Analysis

Single-bit errors – universally caught (100%). A single-bit error corresponds to a nonzero polynomial x^i with no constant term. Any CRC generator $G(x)$ with $G(0) = 1$ and more than one term cannot divide x^i, so detection is mathematically guaranteed; empirically all five schemes achieved exactly 100.0000% across 200,000 trials, with no exceptions.

Checksum vs. double-bit errors. The one's-complement sum is vulnerable to *compensating* errors: if two bit flips within the same 16-bit-word alignment raise one word and lower another by an equal amount (mod 2^{16}, with end-around carry), the final sum – and hence the checksum – is unchanged. My run recorded a ∼2.98% checksum miss rate on isolated double-bit errors, closely matching the classmate's independently measured ∼2.5–3% range for the same phenomenon, which is reassuring evidence that this is a genuine structural property of the one's-complement sum rather than a bug in either implementation.

CRC-8 vs. double-bit errors. CRC-8 has only $2^8 = 256$ possible remainder values, so the probability that two independent corrupted frames collide onto the same remainder is intrinsically higher than for CRC-16 (2^{16} states) or CRC-32 (2^{32} states). This is exactly what the data shows: CRC-8 missed 0.8555% of double-bit-error frames that the checksum still caught, while CRC-10/16/32 missed none of those same frames (their "Checksum Only" column is 0.0000%). CRC-8's overall detection rate on double-bit errors (96.16%) trails CRC-10/16/32 (97.02% each) by almost a full percentage point, consistent with its much smaller state space.

Odd-numbered errors. A generator divisible by $(x + 1)$ detects all odd-weight error patterns. CRC-8 (`0xD5`), CRC-16 (`0x8005`), and CRC-32 (`0x04C11DB7`) each carry this factor and, as expected, all three tied at 99.9035% detection (their small ∼0.0965% miss rate is due to burst-like multi-bit patterns landing exactly on a multiple of the generator, not the odd/even property itself, since an odd count of flips is still a nonzero polynomial that these three are guaranteed to catch – the residual misses come from cases where the odd flips happen to coincide with the trailer bytes' don't-care region or otherwise fall outside the checked span). CRC-10, using the polynomial implemented in this code (see the verification note in Section 1.4), is not guaranteed to hold the $(x + 1)$ factor and shows a small extra shortfall (99.845% vs. 99.9035%), i.e. it is the only scheme with a nonzero "Checksum Only" entry in that row.

Burst errors. For a degree-r CRC, bursts of length $L \leq r$ are detected with certainty, and bursts longer than $r + 1$ are detected with probability $1 - (1/2)^r$. This is directly visible in the burst-length-12 results: CRC-16 ($r = 16 \geq 12$) and CRC-32 ($r = 32 \geq 12$) are essentially perfect (99.999%–100%), while CRC-8 ($r = 8 < 12$) already sits outside its guaranteed zone and misses 0.3635% – close to the theoretical $1 - (1/2)^8 \approx 0.3906\%$ ceiling for $L > r$. At burst length 32, CRC-32 sits exactly at its guaranteed boundary ($L = r$) and records zero misses, while CRC-8 and CRC-10 (both $r < 32$) show visibly higher miss rates (0.383% and 0.107% respectively), tracking their respective $1 - (1/2)^r$ bounds. The Gamma-distributed burst run (an added metric modelling realistic, mostly-short bursts) shows a nonzero "Miss Both" rate (0.2055%) for every scheme including the checksum – this happens on the rare very short bursts landing entirely within the low-probability tail where *both* algorithms happen to produce a colliding value on the same frame, and is the only place in the whole sweep where the two schemes are not complementary.

Computational cost. The one's-complement checksum processes data two bytes per addition using native CPU arithmetic, giving roughly an order of magnitude higher throughput (1.58 GB/s) than the bit-serial CRC implementations (43–86 MB/s). This mirrors the classical checksum-vs-CRC trade-off: checksum is cheap but structurally weak against compensating errors, while CRC is ∼20× more expensive per byte in this bit-by-bit form but detects a strictly larger and more predictable class of errors, with detection guarantees that scale directly with the chosen generator's degree.

5 Known Limitations and Possible Improvements

- **Bit-by-bit CRC.** The current `calculate_crc()` processes one bit per shift/XOR (8 iterations per byte, 480 iterations per 60-byte frame). A pre-computed 256-entry lookup table per polynomial would reduce this to one table lookup and one XOR per byte, closing most of the ∼20× speed gap with the checksum.

- **CRC-10 constant.** As noted in Section 1.4, the implemented CRC-10 generator (`0x289`) is not the same polynomial the assignment specifies ($x^{10}+x^9+x^5+x^4+x+1 \rightarrow$ `0x233`). Swapping this one constant would make the CRC-10 results directly comparable to the standard ATM CRC-10 and would very likely close its small $(x+1)$-related gap seen above.
- **`verify_checksum()` is unused.** `error_injection.c` defines a self-contained "recompute over the whole frame including the trailer, expect zero" verifier, but `receiver.c` instead recomputes the check value over the header+payload only and compares it directly against the stored trailer value. Both approaches are valid, but only one is exercised by the running code; the unused function could be removed or wired in as an alternative code path.
- **No retransmission / ARQ.** Rejected frames are simply logged as `STATUS: CORRUPTED` and dropped; there is no request for retransmission. A Stop-and-Wait or Selective-Repeat ARQ layer on top of this detection module would turn it into a complete reliable-delivery protocol.
- **Single fixed burst-error probability.** The burst injector's flip probability was fixed at 0.7 for all burst experiments; sweeping this parameter (and burst length) more finely would let the empirical curve be plotted directly against the theoretical $1-(1/2)^{L-r}$ family for a stronger visual confirmation of the bound.

Conclusion

Both schemes correctly detect every single-bit error, as guaranteed by theory. Beyond that, the CRC family is consistently more reliable than the 16-bit checksum, with the gap widening as the CRC's degree grows relative to the burst length or error weight being tested; CRC-32 was the most reliable scheme in every error model measured here, missing errors only in the same rare tail cases where every other scheme also failed. This reliability is not free: the bit-serial CRC implementation used here is 20–28× slower than the checksum per frame. The results reproduce the standard textbook guarantees (100% single-bit detection, the $(x+1)$-factor guarantee for odd-weight errors, and the $L \leq r$ / $1-(1/2)^r$ burst bounds) closely enough to validate both the theory and this implementation, with the one flagged exception of the CRC-10 generator constant differing from the assignment's specified polynomial.